

# Documentación Sprint 0

## Aplicación Android

### **Aplicación: DetBLE**

Versión inicial de la aplicación Android encargada de escanear dispositivos Bluetooth Low Energy (iBeacons) y enviar sus mediciones al servidor mediante la API REST.

Desarrollada en **Java**, utilizando las librerías nativas de Android para Bluetooth LE y comunicación HTTP.

**Autor:** Manuel Pérez García

# ÍNDICE

|                                           |          |
|-------------------------------------------|----------|
| <b>1.- Introducción.....</b>              | <b>3</b> |
| <b>2.- Arquitectura general.....</b>      | <b>4</b> |
| <b>3.- Clases principales.....</b>        | <b>5</b> |
| 3.1.- MainActivity.....                   | 5        |
| 3.2.- LogicaAPI.....                      | 6        |
| 3.3.- TramaBeacon.....                    | 6        |
| <b>4.- Flujo de funcionamiento.....</b>   | <b>7</b> |
| <b>5.- Permisos y compatibilidad.....</b> | <b>8</b> |
| <b>6.- Conclusión.....</b>                | <b>8</b> |

# 1.- Introducción

Esta aplicación Android permite detectar tramas **iBeacon** mediante Bluetooth LE, procesar su información y enviarla automáticamente a la API REST del servidor.

La app está pensada para funcionar en conjunto con el servidor Node.js documentado en la API REST (Sprint 0).

El flujo de trabajo principal es el siguiente:

1. El usuario introduce la **IP del servidor**.
2. La app busca dispositivos BLE cercanos.
3. Cuando detecta un iBeacon, obtiene sus valores **major** y **minor**.
4. El campo **major** se divide en dos partes:
  - **Tipo de medición** (byte alto).
  - **Contador** (byte bajo).
5. El valor **minor** se interpreta como el valor medido.
6. La app envía estos datos en formato JSON al servidor mediante un **POST /mediciones**.

## 2.- Arquitectura general

### **MainActivity:**

Actividad principal que gestiona la interfaz, el escaneo Bluetooth y la interacción con el usuario.

### **LogicaAPI:**

Clase auxiliar encargada de construir el cuerpo JSON y realizar las peticiones HTTP (GET/POST) al servidor.

### **TramaBeacon:**

Clase encargada de decodificar la trama iBeacon, extrayendo los campos UUID, Major, Minor y TxPower.

### **Utilidades (externa):**

Clase auxiliar destinada a convertir bytes en enteros o cadenas legibles (por ejemplo, en formato hexadecimal).

### **ApiFormActivity (futura extensión):**

Actividad secundaria planificada para la gestión visual de usuarios o mediciones, sirviendo como interfaz gráfica de la API.

## 3.- Clases principales

### 3.1.- MainActivity

Clase principal de la aplicación. Controla la interfaz, permisos, Bluetooth LE y envío de datos.

#### Funciones destacadas:

- `onCreate()`:  
Inicializa los elementos visuales, listeners y permisos Bluetooth.
- `inicializarBlueTooth()`:  
Verifica permisos y activa el adaptador Bluetooth si está deshabilitado.
- `buscarTodosLosDispositivosBTLE()`:  
Inicia el escaneo BLE general, mostrando información de todos los dispositivos detectados.
- `buscarEsteDispositivoBTLE(String nombre)`:  
Escanea filtrando por nombre específico (introducido por el usuario).
- `mostrarInformacionDispositivoBTLE(ScanResult)`:  
Muestra y decodifica la trama iBeacon recibida.
- `actualizarValoresIBeacon(int major, int minor)`:  
Separa el `major` en tipo y contador, muestra los datos y realiza envío automático si el checkbox está marcado.
- `enviarDatos()`:  
Construye el JSON, obtiene hora y localización, y llama a `LogicaAPI.postMedicion()`.
- `botonEnviarPulsado()`:  
Envía manualmente la medición si el modo automático está desactivado.

#### Elementos de interfaz principales:

- `EditText editTextEntrada`: IP del servidor.
- `EditText dispbusqueda`: nombre del dispositivo BLE a buscar.
- `TextView textValorMajor, textValorMinor`: muestran los valores decodificados.
- `CheckBox checkBoxConfirmacion`: activa/desactiva el envío automático.
- `Button botonEnviar`: envío manual de mediciones.

#### Permisos utilizados:

- `BLUETOOTH_SCAN`
- `BLUETOOTH_CONNECT`
- `ACCESS_FINE_LOCATION`

### 3.2.- LogicaAPI

Clase encargada de la comunicación HTTP con la API REST del servidor.

#### Métodos principales:

- `buildJsonBody(tipomedicion, contador, medicion, user, hora, localizacion):`  
Construye el cuerpo JSON que se envía al servidor en un POST.
- `postMedicion(ip, jsonBody):`  
Envía los datos de una medición a `/mediciones` usando POST.
- `getMediciones(ip):`  
Obtiene todas las mediciones registradas mediante un GET.
- `sendRequest(url, method, jsonBody):`  
Método interno que gestiona la conexión HTTP con manejo de errores y logs.

### 3.3.- TramaIBeacon

Clase utilizada para interpretar los bytes recibidos en una trama iBeacon y extraer los campos relevantes.

#### Campos principales:

- `uuid`: Identificador único del iBeacon (16 bytes).
- `major`: Valor de 2 bytes que contiene tipo y contador.
- `minor`: Valor de 2 bytes que representa la medición.
- `txPower`: Potencia de transmisión.

### Funciones:

- **getUUID():**  
Devuelve el identificador UUID del iBeacon en formato de bytes.
- **getMajor() / getMinor():**  
Devuelven los bytes correspondientes a los campos *Major* y *Minor* del iBeacon.
- **getTxPower():**  
Devuelve la potencia de transmisión (TxPower) del beacon, utilizada para estimar la distancia.
- **TramaIbeacon(byte[] bytes):**  
Constructor que recibe la trama completa en bytes, corrige y almacena los datos recibidos, añadiendo los flags necesarios si la trama no los incluye.

## 4.- Flujo de funcionamiento

1. El usuario abre la aplicación y concede permisos de Bluetooth y localización.
2. La app inicia el escaneo BLE (general o filtrado).
3. Cuando se detecta un iBeacon:
  - Se extraen los bytes **major** y **minor**.
  - Se dividen los bytes del **major** en:
    - Byte alto → tipo de medición.
    - Byte bajo → contador.
  - Se muestra por pantalla:  
**Major:** (tipo | contador)  
**Minor:** valor medido.
4. Si el envío automático está activado (**CheckBox**), se envía la medición.
5. Si está desactivado, el usuario puede pulsar el botón **Enviar**.
6. La clase **LogicaAPI** realiza la petición HTTP POST al servidor en segundo plano.

## 5.- Permisos y compatibilidad

### Permisos requeridos:

- `android.permission.BLUETOOTH_SCAN`
- `android.permission.BLUETOOTH_CONNECT`
- `android.permission.ACCESS_FINE_LOCATION`

### Compatibilidad mínima:

- Android 12 (API 31) o superior (por requerir permisos de BLE modernos).
- Conexión a red WiFi local (misma red que el servidor Node.js).

## 6.- Conclusión

La aplicación Android cumple los siguientes objetivos del Sprint 0:

1. Detectar dispositivos iBeacon mediante Bluetooth LE.
2. Procesar los valores `major` y `minor` como tipo, contador y medición.
3. Obtener la localización actual y la hora del sistema.
4. Enviar las mediciones a la API REST mediante peticiones HTTP POST.
5. Permitir envío manual o automático de datos según configuración del usuario.

Esta estructura base permite futuras ampliaciones, como:

- Registro de usuarios desde la app.
- Visualización de histórico de mediciones.
- Conexión segura mediante HTTPS.
- Soporte para múltiples tipos de sensores BLE.